Intro to Systems Software

CS 449

Final Review

Topics

- *Buffer overflow*
- *Linking (high level understanding)*
- Cache memories
- Virtual Memory
- Memory Allocation
- Processes

Just for fun #1: What does this code print?

Only works with "gcc" ☺

```
#include <stdio.h>
#include <stddef.h>

int foo(int x, int y) {
  struct s {
    char a[x];
    char b[y];
    char c;
  };
  return offsetof(struct s , c);
}

int main(void)
{
   printf("%d\n", foo(2,3));
}
```

Just for fun #2: What does this code print?

Only works with "gcc" ☺

```
#include <stdio.h>
#include <stddef.h>

int bar(int x, int y) {
  struct s {
    char a[x][y];
  };
  return sizeof(struct s);
}

int main(void)
{
   printf("%d\n", bar(2,3));
}
```

Buffer Overflow

Stack Discipline

- This kind of problem covers a wide range of topics, such as stack frames, string representations, ASCII code, and byte ordering
- It demonstrates the dangers of out-of-bounds memory references and the basic ideas behind buffer overflow

Reference

- Read: CSAPP3e 3.10.3 ("Out-of-Bounds Memory References and Buffer Overflow")
- Exercise: CSAPP practice problem 3.46

Practice problem 3.46

- Below is a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result

```
/* This is very low-quality code. It is intended to illustrate
bad programming practices. See Practice Problem 3.46. */
char *get_line()
{
        char buf[4];
        char *result;
        gets(buf);
        result = malloc(strlen(buf));
        strcpy(result, buf);
        return result;
}
```

Practice problem 3.46

- Disassembly up through call to `gets`

```
        char *get_line()
1       0000000000400720 <get_line>:
2       400720:53                       push   %rbx
3       400721:48 83 ec 10              sub    $0x10,%rsp
        See diagram stack at this point
4       400725:48 89 e7                 mov    %rsp,%rdi
5       400728:e8 73 ff ff ff           callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

Practice problem 3.46

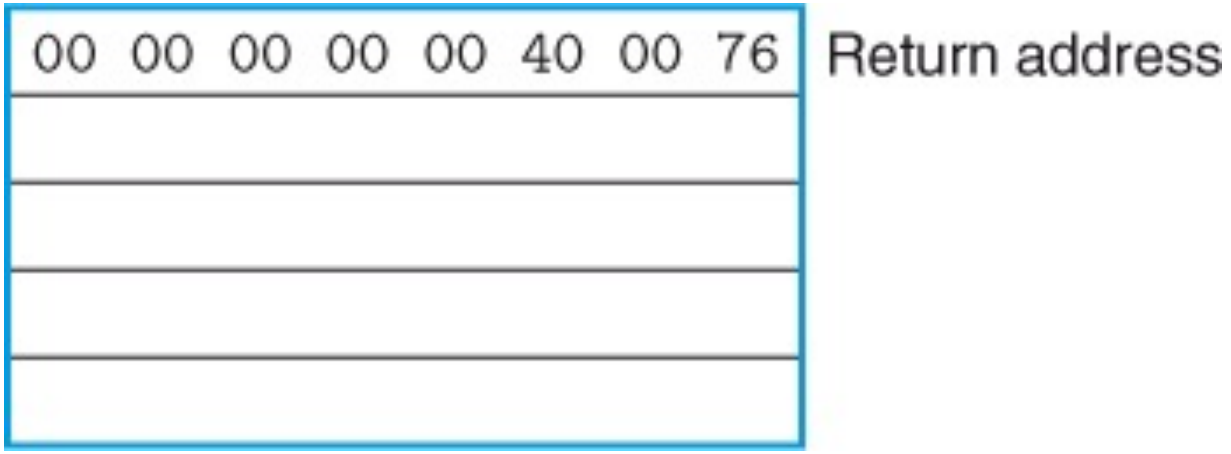
Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string:

```
0123456789012345678901234
```

The program terminates with a **segmentation fault**. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Practice problem 3.46

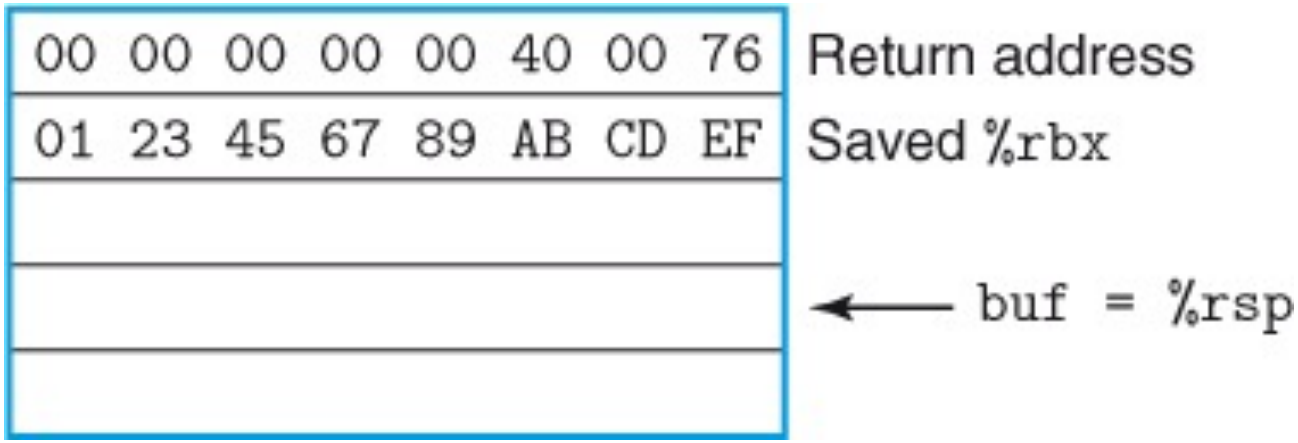
- Question A: Fill in the diagram that follows, indicating as much as you can about the stack just **after** executing the instruction **at line 3 in the disassembly**. Label the quantities stored on the stack (e.g., "Return address") on the right, and their hexadecimal values (if known) within the box. **Each box represents 8 bytes**. Indicate the position of %rsp. Recall that the ASCII codes for characters 0–9 are 0x30–0x39.



Practice problem 3.46

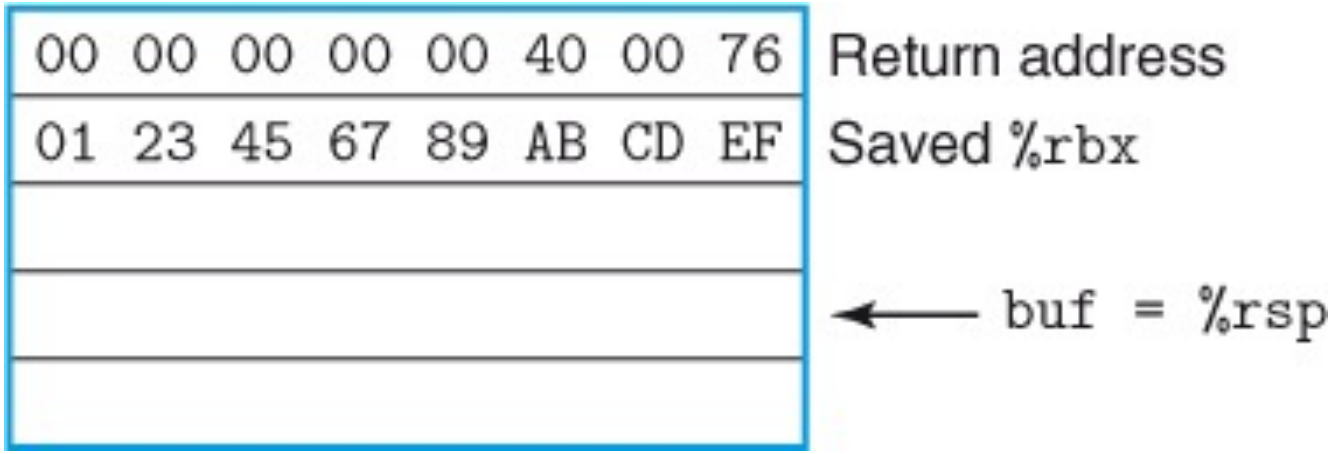
- Question A: Fill in the diagram that follows, indicating as much as you can about the stack just **after** executing the instruction **at line 3 in the disassembly**. Label the quantities stored on the stack (e.g., "Return address") on the right, and their hexadecimal values (if known) within the box. **Each box represents 8 bytes**. Indicate the position of %rsp. Recall that the ASCII codes for characters 0–9 are 0x30–0x39.

Stack after line 3:



Practice problem 3.46

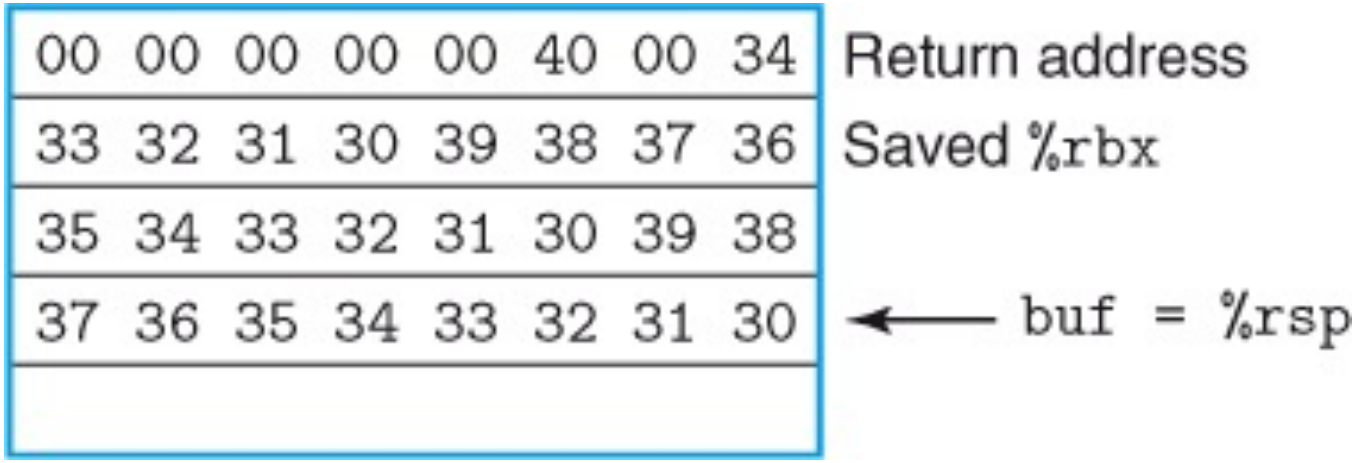
- Question B: Modify your diagram to show the effect of the call to `gets` (line 5)



Practice problem 3.46

- Question B: Modify your diagram to show the effect of the call to `gets` (line 5)

Stack after line 5:



Practice problem 3.46

- Question C: To what address does the program attempt to return?

The program is attempting to return to address `0x040034`. The low-order 2 bytes were overwritten by the code for character '4' and the terminating null character.

Practice problem 3.46

- Question D: What register(s) have corrupted value(s) when `get_line` returns?

The saved value of register `%rbx` was set to `0x3332313039383736`.
This value will be loaded into the register before `get_line` returns.

Practice problem 3.46

- Question E: Besides the potential for buffer overflow, what two other things are wrong with the code for function `get_line`?

The call to malloc should have had `strlen(buf)+1` as its argument, and the code should also check that the returned value is not equal to `NULL`.

Cache Memories

Practice Cache Problem

- We have a 64 KiB address space
- The cache is a 1 KiB, direct-mapped cache using 256-byte blocks with *write-back* and *write-allocate* policies

- **write-allocate:** memory is read after a cache-miss-on-write
- **write-back:** dirty data is written back to memory on eviction

a) Calculate the TIO address breakdown:

Tag	Index	Offset

Practice Cache Problem

- We have a 64 KiB address space
- The cache is a 1 KiB, direct-mapped cache using 256-byte blocks with *write-back* and *write-allocate* policies

- **write-allocate:** memory is read after a cache-miss-on-write
- **write-back:** dirty data is written back to memory on eviction

a) Calculate the TIO address breakdown:

Tag	Index	Offset
$16 - 2 - 8 = \mathbf{6}$	$\frac{Cache}{Block} = \frac{2^{10}}{2^8} = 2^2 \rightarrow 2$	$2^8 = 256 \rightarrow 8$

Practice Cache Problem

b) During some part of a running program, the cache's management bits are as shown below.

Line	Valid	Dirty	Tag
00	0	0	1000 01
01	1	1	0101 01
10	1	0	1110 00
11	0	0	0000 11

Four options for the next two memory accesses are given (R = read, W = write). Circle the option that results in data from the cache *being written to memory*

Practice Cache Problem

Line	Valid	Dirty	Tag
00	0	0	1000 01
01	1	1	0101 01
10	1	0	1110 00
11	0	0	0000 11

(1) R 0x4C00, W 0x5C00

(2) W 0x5500, W 0x7A00

(3) W 0x2300, R 0x0F00

(4) R 0x3000, R 0x3000

Practice Cache Problem

Line	Valid	Dirty	Tag
00	0	0	1000 01
01	1	1	0101 01
10	1	0	1110 00
11	0	0	0000 11

(1) R 0x4C00, W 0x5C00

R 0b0100 1100… , W 0b0101 1100…

(2) W 0x5500, W 0x7A00

W 0b0101 0101… , W 0b0111 1010…

(3) W 0x2300, R 0x0F00

W 0b0010 0011… , R 0000 1111…

(4) R 0x3000, R 0x3000

R 0b0011 0000… , R 0011 0000…

Practice Cache Problem

Line	Valid	Dirty	Tag
00	0	0	1000 01
01	1	1	0101 01
10	1	0	1110 00
11	0	0	0000 11

(1) R 0x4C00, W 0x5C00

R 0b0100 1100… , W 0b0101 1100…

The read evicts line 0, but the dirty bit was not set so nothing is written (also, line 0 was initially invalid). The write overwrites line 0 again but since the cache is write-back nothing is written to memory.

(2) W 0x5500, W 0x7A00

W 0b0101 0101… , W 0b0111 1010…

The first write doesn't evict anything because the tags match. The second write evicts the old data, but the dirty bit was not set so the old data doesn't need to be written back to memory.

(3) W 0x2300, R 0x0F00

W 0b0010 0011… , R 0000 1111…

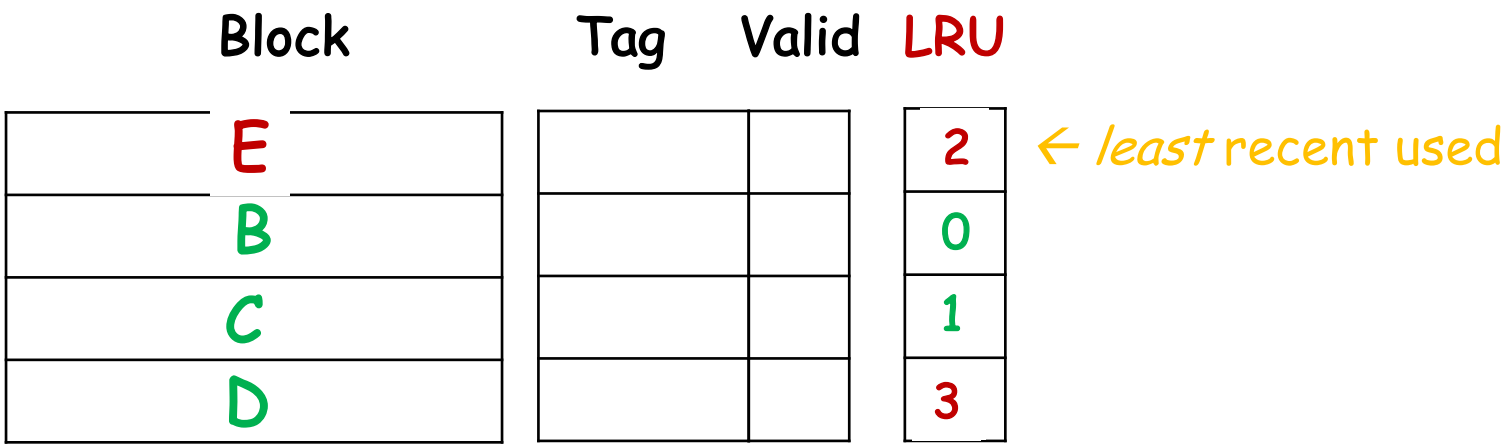
The write evicts line 3 which was invalid and not dirty, so nothing is written to memory. The read, however, also maps to line 3 so it must write the value changed in the write back to memory before it can update the cache.

(4) R 0x3000, R 0x3000

R 0b0011 0000… , R 0011 0000…

Line 0 is initially not dirty (and invalid) so nothing is written back to memory from either of these reads (which both read from the same line).

Implementing LRU (Least Recently Used)



Block requests: E D

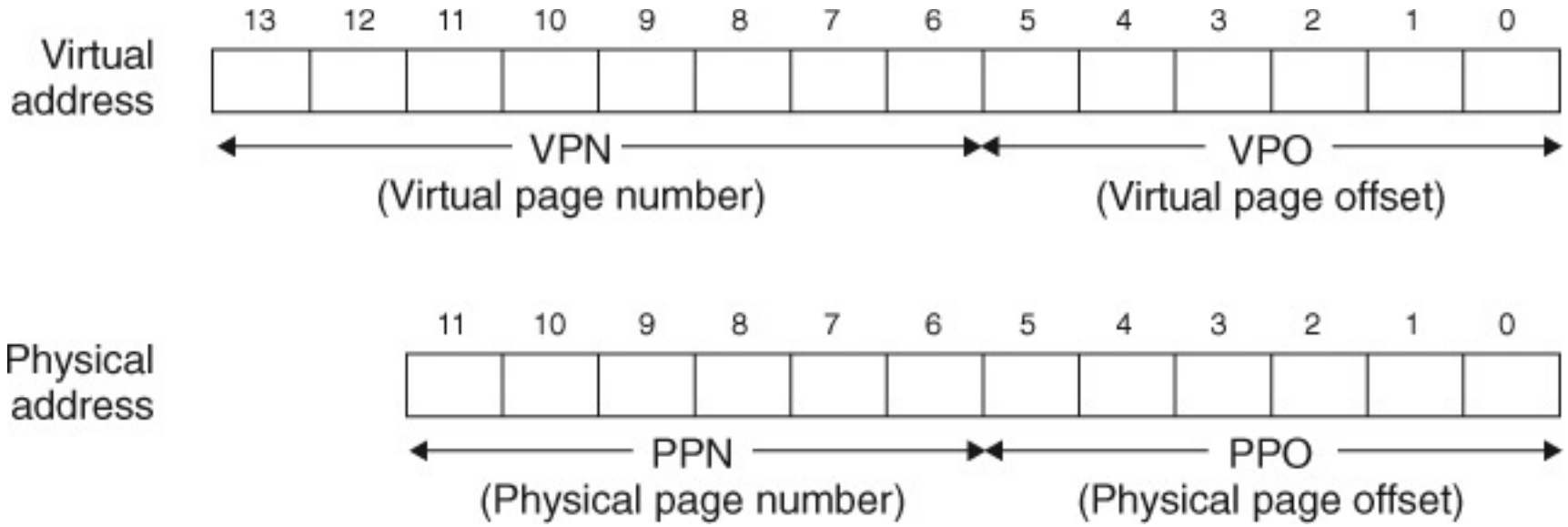
How large (in bits) should the LRU counter be on an N-way set-associative cache?

Virtual Memory

End-to-End Address Translation

- Consider the following memory system:
  - The memory is *byte addressable*
  - Virtual addresses are 14 bits wide ($n = 14$)
  - Physical addresses are 12 bits wide ($m = 12$)
  - The page size is 64 Bytes ($P = 64$)
  - The L1 d-cache is physically addressed and direct mapped, with a 4-Byte line size and 16 total sets

Virtual and physical addresses



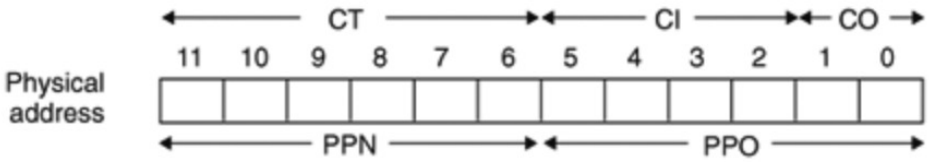
- 14-bit virtual addresses (n = 14)
- 12-bit physical addresses (m = 12)
- 64-byte pages (P = 64)

Snapshot of the page table

VPN	PPN	Valid
00	28	1
01	—	0
02	33	1
03	02	1
04	—	0
05	16	1
06	—	0
07	—	0

VPN	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	—	0
0C	—	0
0D	2D	1
0E	11	1
0F	0D	1

Snapshot of the L1 d-cache



Idx	Tag	Valid	Blk 0	Blk 1	Blk 2	Blk 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	08
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

4B-block

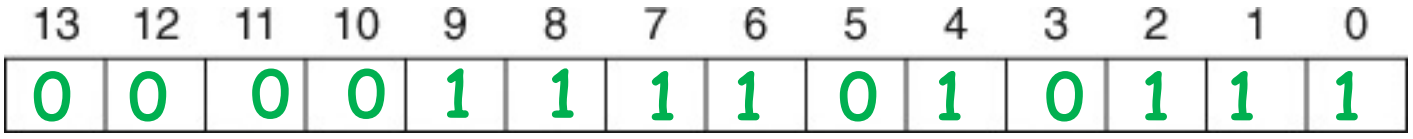
(c) Cache: 16 sets, 4-byte blocks, direct mapped

Practice Problem 9.4 (CSAPP:3e)

- Show how the example memory system translates a virtual address into a physical address and accesses the cache

- Given virtual address: **`0x03d7`**

What is the virtual address format?



Address Translation

Parameter	Value
VPN	0x0f
Page fault? (Y/N)	N
PPN	0x0d

Bit position	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VA: 0x03d7	0	0	0	0	1	1	1	1	0	1	0	1	1	1
	VPN								VPO					
	0x0f								0x17					

Physical Memory Access

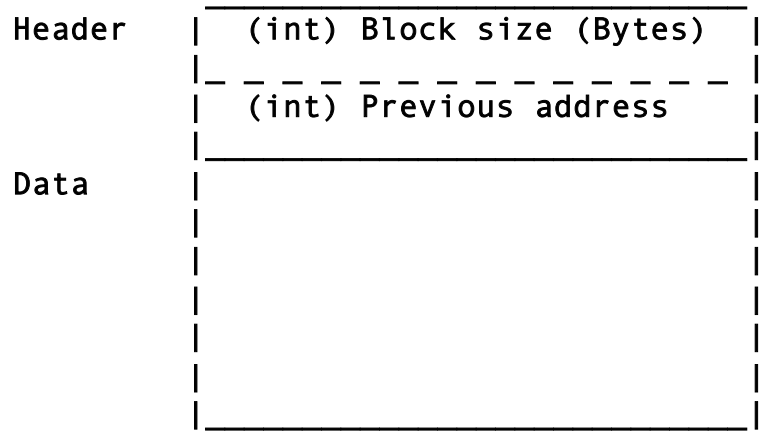
Parameter	Value
Byte offset	0x03
Cache index	0x05
Cache tag	0x0d
Cache hit? (Y/N)	Y
Cache byte returned	0x1d

	CT						CI				CO	
	0x0d						0x05				0x03	
Bit position	11	10	9	8	7	6	5	4	3	2	1	0
PA : 0x03d7	0	0	1	1	0	1	0	1	0	1	1	1
	PPN						PPO					
	0x0d						0x17					

Memory Allocation

Dynamic storage allocation

- Consider an allocator that uses an list. The layout of each allocated AND free memory block is as follows:
- Each memory block, either allocated or free, has a size.
- The size doesn't include the header.
- The size is negative if the block is free, positive otherwise
- It's a 32-bit machine, addresses and ints are both 32-bits



- Given the contents of the heap shown on the left, show the new contents of the heap (in the right table) after a call to **`free(0x400b014)`** is executed
- Your answers should be given as hex values
- Note that the address grows from bottom up
- Assume that the allocator uses immediate coalescing, that is, adjacent free blocks are merged immediately each time a block is freed



Processes

Processes

- Consider this code using Linux's `fork`:

```
int x = 7;
if ( fork() ) {
   x++;
   printf(" %d ", x);
   fork();
   x++;
   printf(" %d ", x);
} else {
   printf(" %d ", x);
}
```

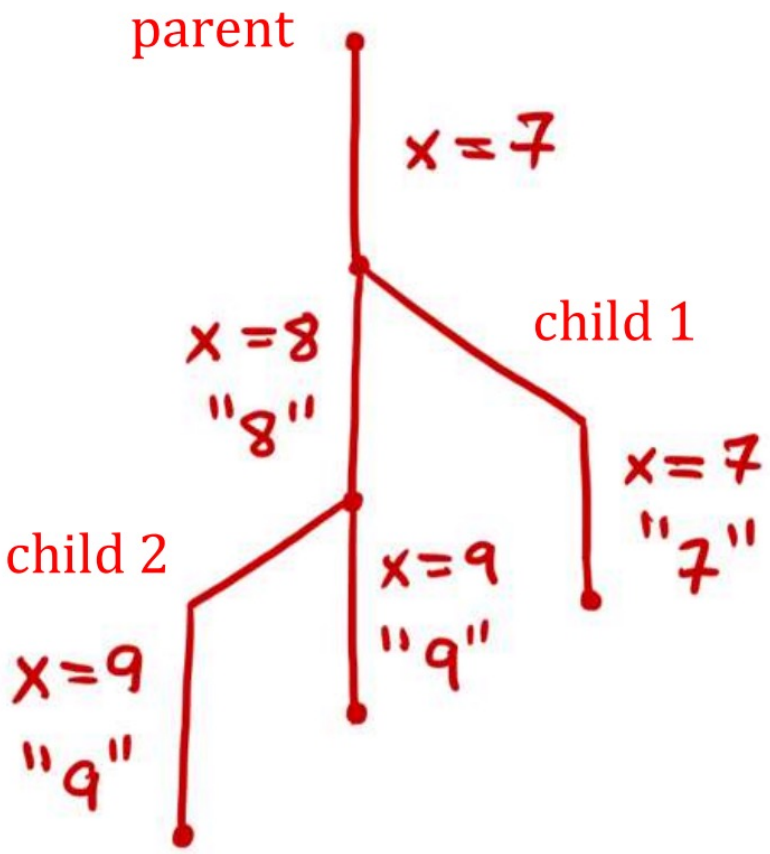
What are all the different possible outputs (i.e. order of things printed) for this code? (Hint: there are 4 of them.)

Tip: try drawing a process graph for this program

Processes

- Consider this code using Linux's `fork`:

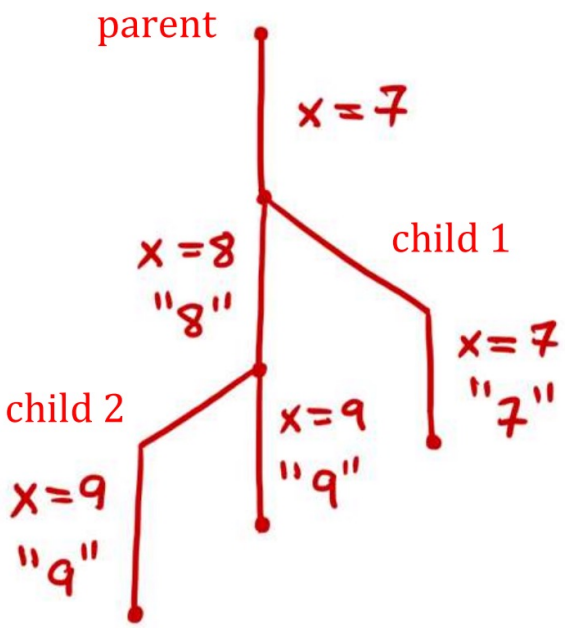
```
int x = 7;
if ( fork() ) {
   x++;
   printf(" %d ", x);
   fork();
   x++;
   printf(" %d ", x);
} else {
   printf(" %d ", x);
}
```



Processes

- Consider this code using Linux's fork:

```
int x = 7;
if ( fork() ) {
   x++;
   printf(" %d ", x);
   fork();
   x++;
   printf(" %d ", x);
} else {
   printf(" %d ", x);
}
```



Possible orderings:

7 8 9 9
8 7 9 9
8 9 7 9
8 9 9 7

More stuff

Dynamic storage allocation

- Five helper routines are defined to facilitate the implementation of `free(void *p)`
- The functionality of each routine is explained in the comment above the function definition
- Fill in the body of the helper routines the code section label that implement the corresponding functionality correctly
- Important notice: Block sizes are 4 bytes in this problem

Dynamic storage allocation

- Code 1:

```
/* given a pointer p to an allocated block, i.e., p is a
   pointer returned by some previous malloc()/realloc() call;
   returns the pointer to the header of the block*/
void * header(void* p)
{
  void *ptr;

  _______;
  return ptr;
}

A. ptr = p-2
B. ptr = (void *)((int *)p-2)
C. ptr = (void *)((int *)p+2)
```

Dynamic storage allocation

- Code 2:

```
/* given a pointer to a valid block header or footer,
   returns the size of the block */
int size(void *hp)
{
  int result;

  _______;
  return result;
}

A. result = (*hp>0)?*hp:-*hp;
B. result = *(int *)hp;
C. result = (*(int *)hp > 0)?(*(int *)hp):(-*(int *)hp);
```

Dynamic storage allocation

- Code 3:

```
/* given a pointer p to an allocated block, i.e. p is
   a pointer returned by some previous malloc()/realloc() call;
   returns the pointer to the next of the block*/
void * next(void *p)
{
  void *ptr;

  _______;
  return ptr;
}
```

A. ptr = p+size(header(p))
B. ptr = p+size(header(p))-8
C. ptr = (int *)p+size(header(p))-2

Dynamic storage allocation

- Code 4:

```
/* given a pointer to a valid block header or footer,
   returns the usage of the current block,
   1 for allocated, 0 for free */
int allocated(void *hp)
{
  int result;

  ______;
  return result;
}
```

A. result=(*(int *)hp)>0
B. result=(*(int *)hp)==0
C. result=(*(int *)hp)<0

Dynamic storage allocation

- Code 5:

```
/* given a pointer to a valid block header,
   returns the pointer to the header of previous block in
memory */
void * prev(void *hp)
{
  void *ptr;

  _______;
  return ptr;
}
```

A. `ptr = hp - *((int*)hp + 1)`

B. `ptr = (void*)*((int*)hp + 1)`

C. `ptr = (void*)((int*)hp + 1)`

Cache Puzzle

- Given the following sequence of access results (addresses are given in decimal) on a cold/empty cache of size 16 bytes, what can we deduce about its properties? *Assume an LRU replacement policy*

```
(0, Miss), (8, Miss), (0, Hit), (16, Miss), (8, Miss)
```

What can we say about the *block size*?

Cache Puzzle

- Given the following sequence of access results (addresses are given in decimal) on a cold/empty cache of size 16 bytes, what can we deduce about its properties? *Assume an LRU replacement policy*

```
(0, Miss), (8, Miss), (0, Hit), (16, Miss), (8, Miss)
```

What can we say about the *block size*?

The block size must be ≤ 8 because access (2) to address 8 is a miss after access (1) to address 0 is a hit.

Cache Puzzle

`(0, Miss), (8, Miss), (0, Hit), (16, Miss), (8, Miss)`

Assuming that the block size is 8 bytes, can this cache be… (Hint: draw the cache and simulate it)

a. Direct-mapped?

Does this cache work for the access results? Yes, Yes, Yes, Yes (evict 0), No (8 would still be in cache)

Cache Puzzle

`(0, Miss), (8, Miss), (0, Hit), (16, Miss), (8, Miss)`

Assuming that the block size is 8 bytes, can this cache be… (Hint: draw the cache and simulate it)

b. 2-way set associative?

Does this cache work for the access results? Yes, Yes, Yes, Yes (evict 8 b/c it's the least recently used), Yes (8 is no longer in cache)

Cache Puzzle

`(0, Miss), (8, Miss), (0, Hit), (16, Miss), (8, Miss)`

Assuming that the block size is 8 bytes, can this cache be… (Hint: draw the cache and simulate it)

c. 4-way set associative?

No, because the block size is 8, multiplied by 4 lines per set, and that's 32B, which is already bigger than the entire cache.

Code Analysis Problem

- Assuming the cache starts cold (all blocks invalid) and `sum`, `i`, and `j` are stored in registers, calculate the **miss rate**:
m = 12 bits (addr. range), $Cache$ = 256 B, $Block$ = 32 B, Ass = 2

```
#define SIZE 8
long ar[SIZE][SIZE], sum = 0;   // &ar=0x800
for (int i = 0; i < SIZE; i++)
   for (int j = 0; j < SIZE; j++)
      sum += ar[i][j];
```

- Code employs a row-wise access pattern
- Cache block (of 32 bytes) holds 4 long's
- There is a miss followed by 3 hits (MR=25%)

Code Analysis Problem

- How would the following changes change the miss rate?
  - Increasing the block size?
  - Doubling the cache size
  - Reducing/increasing the associativity